

Plan B — Code Implementation Plan

SIH 2025 · Problem Statement 26085 — Urban Flood Nowcasting System

Team: 6 members · Project name: FloodCast

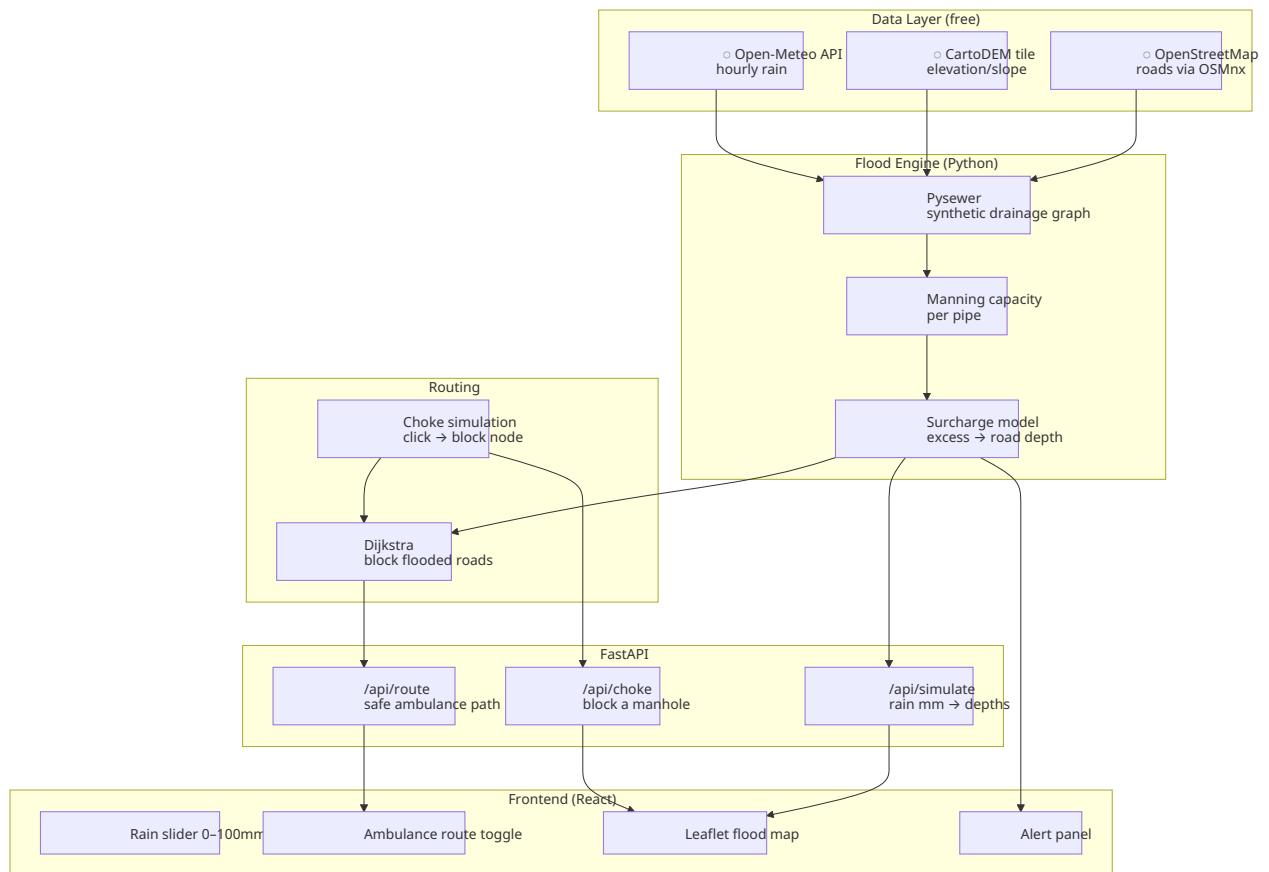
This is the **code/build workstream**. It covers repo structure, module ownership, APIs, the flood engine, and the 36-hour timeline. The presentation/demo workstream is in **Plan A**.

1. Team Role Map (6 members — Code track)

#	Member	Role	Owns these modules	Tech
1	Team Lead / Backend Lead	Architecture + API orchestration	api/, project wiring, Render deploy	FastAPI, Docker
2	GIS / Data Engineer	Rainfall + DEM + drainage data pipeline	data/, pipeline/	rasterio, OSMnx, Pysewer, Open-Meteo
3	Hydraulics Engine Dev	Flood model + pipe capacity + surcharge	engine/	NetworkX, NumPy, Manning's eq
4	Routing Dev	Safe ambulance route + choke simulation	routing/	NetworkX Dijkstra, OSMnx
5	Frontend Dev	3D flood map + rain slider + alerts UI	frontend/	React, Leaflet/Deck.gl, Vite
6	QA / Alerts / Deploy	Testing, Telegram bot, deployment CI	tests/, alerts/	pytest, python-telegram-bot, GitHub Actions

Each member pushes to a clearly named branch; merges happen via PR. Lead merges.

2. System Architecture (one-picture)



3. Repository Structure

```
floodcast/  
├── api/ # FastAPI app (Backend Lead)  
│   ├── main.py # routes: /simulate /route /choke /health  
│   └── schemas.py # Pydantic request/response models  
├── engine/ # Hydraulics Engine Dev  
│   ├── graph_build.py # build NetworkX graph from drainage + DEM  
│   ├── capacity.py # Manning's equation → pipe Q_cap  
│   └── surcharge.py # inflow>capacity → road water depth (cm)  
├── routing/ # Routing Dev  
│   ├── safe_route.py # Dijkstra with flooded nodes removed  
│   └── choke.py # block a node → recompute  
├── data/ # GIS Data Engineer  
│   ├── rain.py # Open-Meteo fetch + cache JSON  
│   ├── dem.py # load CartoDEM tile, compute slope  
│   ├── drainage.py # Pysewer/OSMnx → synthetic pipe graph  
│   └── cache/ # pre-downloaded DEM tile + rain JSON (offline demo)  
├── alerts/ # QA/Alerts  
│   └── telegram_bot.py # auto flood alert to bot  
├── frontend/ # Frontend Dev  
│   └── src/  
│       └── App.jsx # rain slider + map + route toggle
```

```

| | └─ FloodMap.jsx # Leaflet/Deck.gl depth colours
| | └─ api.js # calls FastAPI
| └─ vite.config.js
└─ tests/ # QA
    │ └─ test_engine.py
    │ └─ test_routing.py
    │ └─ test_api.py
└─ docker-compose.yml
└─ requirements.txt
└─ README.md

```

4. Core Algorithms (the actual model)

4a. Build the drainage graph (GIS → Engine)

```

# data/drainage.py
import osmnx as ox, networkx as nx
from pysewer import network, routing # synthetic sewer generation
def build_drainage_graph(place_point, dist=1500):
    roads = ox.graph_from_point(place_point, dist=dist, network_type="drive")
    # manholes = road intersections (nodes), pipes = road segments (edges)
    G = nx.Graph()
    for u, v, d in roads.edges(data=True):
        G.add_edge(u, v,
            length=d.get("length", 50),
            diameter=0.3, # assumed 300 mm pipe (proxy)
            roughness=0.013) # concrete
    # attach elevation from DEM to each node (GIS Engineer)
    # ... rasterio sample at node lat/lon
    return G

```

4b. Pipe capacity — Manning's equation (Engine Dev)

```

# engine/capacity.py
import math
def pipe_capacity(diameter_m, length_m, slope, roughness=0.013):
    r = diameter_m / 2
    A = math.pi * r**2
    P = 2 * math.pi * r
    R_h = A / P
    slope = max(slope, 0.001) # avoid zero slope
    return (1/roughness) * A * (R_h ** (2/3)) * math.sqrt(slope) # m³/s

```

4c. Surcharge → street depth (Engine Dev)

```

# engine/surcharge.py

```

```
def simulate(G, rain_mm_hr, minutes=30):
    depth_cm = {n: 0.0 for n in G.nodes}
    for n in G.nodes:
        area = G.nodes[n].get("catch_area", 2000) # m² road draining here
        inflow = rain_mm_hr * area * 1e-3 / 3.6 # m³/s
        # distribute to downstream pipes by slope capacity
        for nbr in G.neighbors(n):
            e = G[n][nbr]
            q_cap = pipe_capacity(e["diameter"], e["length"], e["slope"])
            taken = min(inflow, q_cap)
            inflow -= taken
            excess = max(0.0, inflow) # pipes couldn't take it
            depth_cm[n] = excess * (minutes*60) / (area * 100)
    return depth_cm
```

4d. Safe ambulance route (Routing Dev)

```
# routing/safe_route.py
import networkx as nx
def safe_route(road_G, depth_cm, source, target, threshold_cm=15):
    G = road_G.copy()
    for n, d in depth_cm.items():
        if d > threshold_cm:
            G.remove_node(n) # block flooded roads
    return nx.shortest_path(G, source, target, weight="length")
```

4e. Choke simulation (Routing Dev)

```
# routing/choke.py
def choke(G, node):
    G.nodes[node]["blocked"] = True # zero its pipes' capacity
    for nbr in G.neighbors(node):
        G[node][nbr]["diameter"] = 0.0
```

5. API Endpoints (Backend Lead)

Method	Route	Input	Output
POST	<code>/api/simulate</code>	<code>{rain_mm: 50, minutes: 30}</code>	<code>{depths: {node_id: cm}}</code>
POST	<code>/api/route</code>	<code>{source, target, threshold_cm: 15}</code>	<code>{path: [...], eta_saved_sec}</code>
POST	<code>/api/choke</code>	<code>{node_id}</code>	<code>{flooded_neighbours: [...]}</code>
GET	<code>/api/rain/live</code>	—	live Open-Meteo rain for default city
GET	<code>/health</code>	—	<code>{status: "ok"}</code> (for keep-alive)

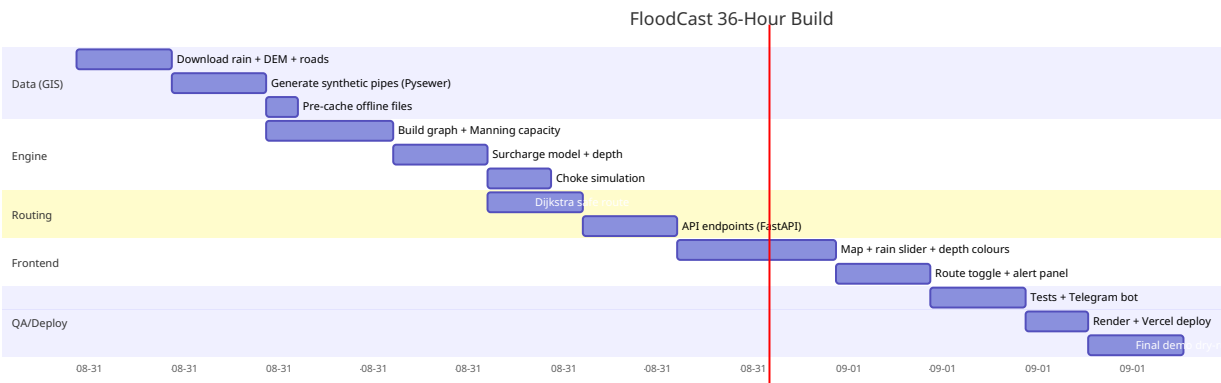
Frontend calls these; all return JSON in <2 sec because the graph is small (~5 km²).

6. Datasets to Download (one-time, before coding starts)

Data	Tool/Link	Owner	Output file
Rain (live)	Open-Meteo API (no key)	GIS	data/cache/rain.json
DEM tile	CartoDEM (Bhuvan) or ALOS PALSAR (ASF Vertex)	GIS	data/cache/dem.tif (5 km²)
Roads	osmnx.graph_from_point	GIS	data/cache/roads.graphml
Synthetic pipes	Pysewer (roads + DEM)	GIS	data/cache/pipes.graphml
Validation	Bhuvan flood layer / Sentinel-1 (Copernicus)	QA	data/validation/

Pre-cache everything so the demo runs **offline** — the #1 hackathon failure is Wi-Fi dying mid-demo.

7. 36-Hour Timeline (parallel tracks)



Member task sheet (who does what, hour by hour)

Member	Hours 0–6	Hours 6–18	Hours 18–30	Hours 30–36
1 Backend Lead	Repo + Docker setup	API scaffolding	Wire engine↔frontend	Deploy + dry-run
2 GIS/Data	Download rain/DEM/OSM	Pysewer pipes + cache	DEM slope attach	Data validation
3 Hydraulics	—	Manning + surcharge	Tune depth formula	Bug-fix depth
4 Routing	—	Dijkstra safe route	Choke sim + API	Tune threshold
5 Frontend	Vite + map skeleton	Rain slider + colours	Route toggle + alerts	Polish UI
6 QA/Deploy	Test plan + CI skeleton	Unit tests engine	Telegram bot	Deploy + keep-alive

8. Testing & Quality (QA)

- Unit tests (`pytest`): `test_capacity` (known pipe → known Q), `test_surcharge` (zero rain → zero depth), `test_route` (blocked node not in path).

- **Integration test:** full `/simulate` → `/route` cycle returns valid JSON in <2 sec.
- **Smoke test script:** `scripts/demo_smoke.py` runs the exact demo flow (slider 50mm → choke → route) so you can verify in one command before the pitch.
- **Accuracy check:** compare predicted flood nodes vs Bhuvan/Sentinel-1 historical flood polygon → report % overlap on slide 4.

9. Deployment (matches your 25105 pattern)

Service	What	Free?
Render web service	FastAPI backend → <code>floodcast-api.onrender.com</code>	✓ (your team used Render before)
Vercel	React frontend → <code>floodcast.vercel.app</code>	✓
UptimeRobot	ping <code>/health</code> every 5 min so Render doesn't sleep	✓
ngrok	backup tunnel from laptop if venue Wi-Fi fails	✓
GitHub Actions	auto-deploy on push to <code>main</code>	✓

10. Jury Q&A — 3 must-have answers (memorise)

1. **"Pipe data kahan se laoge?"** → "Real underground sewer data isn't public in India. We auto-generate a **synthetic proxy drainage network** from OpenStreetMap roads + DEM slope using the **Pysewer** method — validated in recent hydrology research for data-scarce cities. Topology is realistic; pipe diameters are assumed."
2. **"Radar/DEM files are huge — won't your API crash?"** → "We pre-download a small ~5 km² DEM tile + a cached Open-Meteo JSON. The demo runs fully **offline**; live API is a bonus."
3. **"SWMM is slow — how is this real-time?"** → "We replaced SWMM with a **lightweight Manning-capacity check in NetworkX** — runs in ~2 seconds, sufficient for nowcasting. SWMM can be added later for production."

11. Final Delivery Checklist (Plan B)

- ☐ Repo on GitHub with clean README + architecture image
- ☐ All 5 API endpoints working and tested
- ☐ Engine returns depth in <2 sec for the demo area
- ☐ Pre-cached data so demo works offline
- ☐ Frontend deployed on Vercel + backend on Render
- ☐ Public link + QR code (given to Plan A for slide 3)
- ☐ Smoke-test script passes
- ☐ Telegram alert bot working
- ☐ Backup ngrok link ready

Plan B — code/build workstream for SIH 26085 FloodCast. Pair this with Plan A (PPT, screenshots, video, live link). The PPT follows the attached official SIH template (max 6 slides, export as PDF).